Define the Marketing Funnel (First & Most Important Step)

Before touching Python, define funnel stages clearly.

A standard funnel:

Visitors → Leads → Qualified Leads → Customers

```
import pandas as pd
import numpy as np

# Load data
df = pd.read_csv("/content/bank-full.csv",sep=';')

# Quick inspection
df.head()
df.info()
df.isna().sum()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 45211 entries, 0 to 45210
Data columns (total 17 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   age        45211 non-null  int64
 1   job        45211 non-null  object
 2   marital    45211 non-null  object
 3   education  45211 non-null  object
 4   default    45211 non-null  object
 5   balance    45211 non-null  int64
 6   housing    45211 non-null  object
 7   loan       45211 non-null  object
 8   contact    45211 non-null  object
 9   day        45211 non-null  int64
 10  month      45211 non-null  object
 11  duration   45211 non-null  int64
 12  campaign   45211 non-null  int64
 13  pdays      45211 non-null  int64
 14  previous   45211 non-null  int64
 15  poutcome   45211 non-null  object
 16  y          45211 non-null  object
dtypes: int64(7), object(10)
memory usage: 5.9+ MB
```

	0
age	0
job	0
marital	0
education	0
default	0
balance	0
housing	0
loan	0
contact	0
day	0
month	0
duration	0
campaign	0
pdays	0
previous	0
poutcome	0
y	0

dtype: int64

2.1 Data Cleaning & Structuring

```
df.columns = df.columns.str.lower().str.replace(" ", "_")
```

```
# Example: drop rows with critical missing info
df = df.dropna(subset=["job", "education"])
```

```
df["is_visitor"] = 1
df["is_lead"] = np.where(df["contact"] != "unknown", 1, 0)
df["is_customer"] = np.where(df["y"] == "yes", 1, 0)
```

3.1 Funnel Volume Analysis

```
funnel_counts = {
    "Visitors": df["is_visitor"].sum(),
    "Leads": df["is_lead"].sum(),
    "Customers": df["is_customer"].sum()
}

funnel_df = pd.DataFrame.from_dict(
    funnel_counts, orient="index", columns=["count"]
)

funnel_df
```

	count
Visitors	45211
Leads	32191
Customers	5289

Next steps: Generate code with `funnel_df` New interactive sheet

4.1 Conversion Rate Analysis

```
visitor_to_lead = funnel_counts["Leads"] / funnel_counts["Visitors"]
lead_to_customer = funnel_counts["Customers"] / funnel_counts["Leads"]
visitor_to_customer = funnel_counts["Customers"] / funnel_counts["Visitors"]

conversion_df = pd.DataFrame({
    "Metric": [
        "Visitor → Lead",
        "Lead → Customer",
        "Visitor → Customer"
    ],
    "Conversion Rate": [
        visitor_to_lead,
        lead_to_customer,
        visitor_to_customer
    ]
})

conversion_df
```

	Metric	Conversion Rate
0	Visitor → Lead	0.712017
1	Lead → Customer	0.164301
2	Visitor → Customer	0.116985

Next steps: Generate code with conversion_df New interactive sheet

```
channel_perf = (
    df.groupby("contact")
    .agg(
        visitors=("is_visitor", "count"),
        leads=("is_lead", "sum"),
        customers=("is_customer", "sum")
    )
)

channel_perf["lead_conversion"] = channel_perf["leads"] / channel_perf["visitors"]
channel_perf["customer_conversion"] = channel_perf["customers"] / channel_perf["leads"]

channel_perf.sort_values("customer_conversion", ascending=False)
```

contact	visitors	leads	customers	lead_conversion	customer_conversion
unknown	13020	0	530	0.0	inf
cellular	29285	29285	4369	1.0	0.149189
telephone	2906	2906	390	1.0	0.134205

```
channel_perf["drop_after_lead"] = (
    channel_perf["leads"] - channel_perf["customers"]
)

channel_perf[["drop_after_lead"]].sort_values(
    by="drop_after_lead", ascending=False
)
```

contact	drop_after_lead
cellular	24916
telephone	2516
unknown	-530

```
df["date"] = pd.to_datetime(df["day"], errors="coerce")

monthly_trend = (
    df.groupby(df["date"].dt.month)
    .agg(customers=("is_customer", "sum"))
)

monthly_trend
```

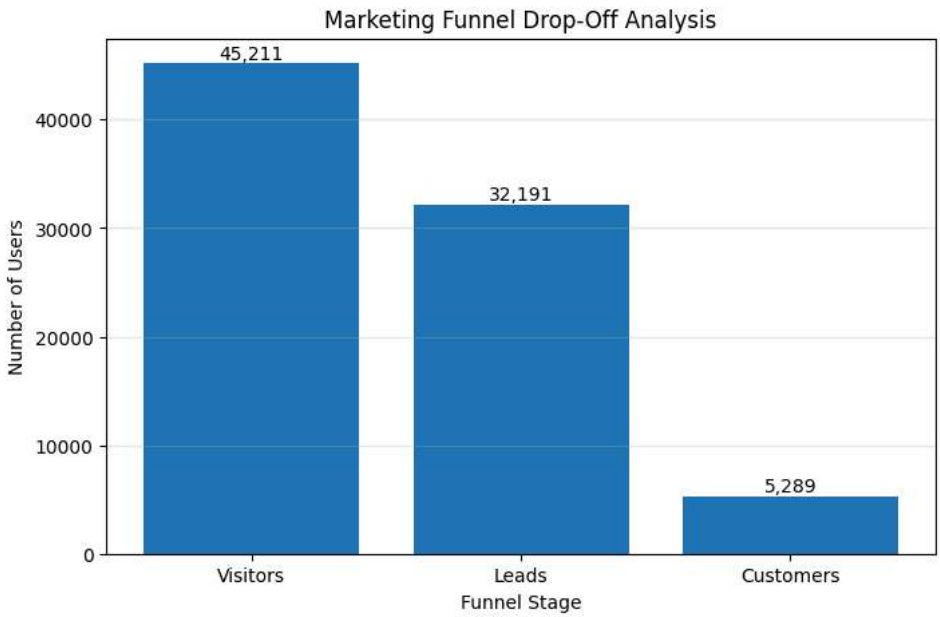
date	customers
1	5289

Chart : Funnel Bar Chart

```
plt.figure(figsize=(8,5))
plt.bar(funnel_df.index, funnel_df["count"])
plt.title("Marketing Funnel Drop-Off Analysis")
plt.xlabel("Funnel Stage")
plt.ylabel("Number of Users")
plt.grid(axis="y", alpha=0.3)

for i, val in enumerate(funnel_df["count"]):
    plt.text(i, val, f"{val:,}", ha="center", va="bottom")
```

```
plt.show()
```



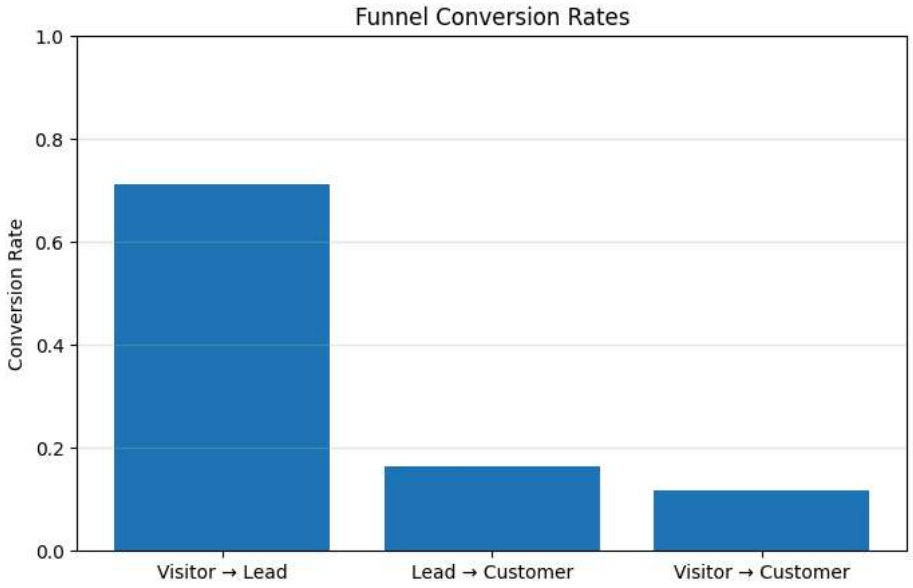
Immediate visibility of largest drop-off

Stakeholders instantly see funnel leakage

Perfect opening chart in any presentation

Chart : Conversion Rate Visualization

```
plt.figure(figsize=(8,5))
plt.bar(conversion_df["Metric"], conversion_df["Conversion Rate"])
plt.title("Funnel Conversion Rates")
plt.ylabel("Conversion Rate")
plt.ylim(0,1)
plt.grid(axis="y", alpha=0.3)
plt.show()
```



💡 Insight:

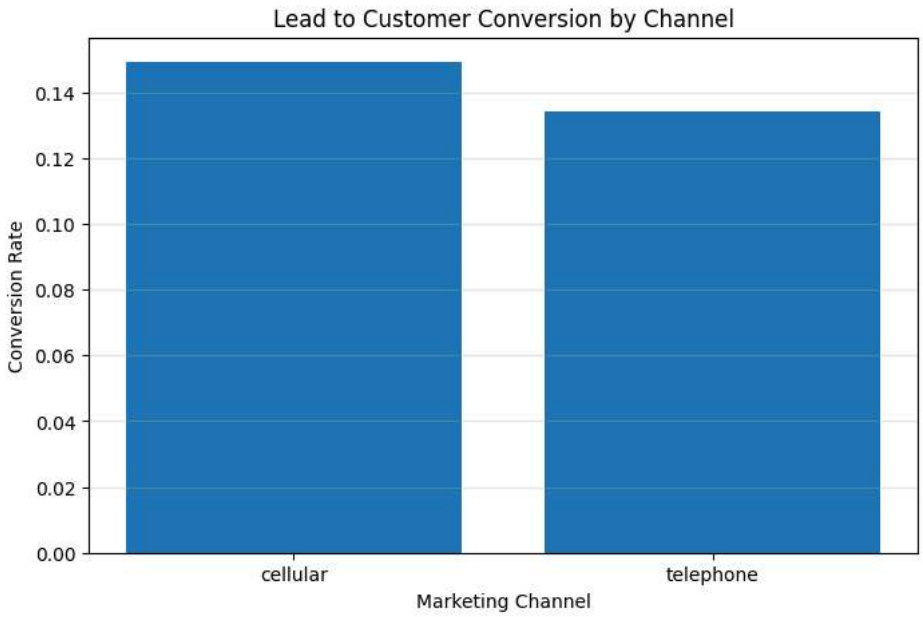
Identifies weakest funnel transition

Helps prioritize optimization efforts

Excellent for KPI reviews

Chart : Channel-wise Funnel Performance

```
plt.figure(figsize=(8,5))
plt.bar(channel_perf.index, channel_perf["customer_conversion"])
plt.title("Lead to Customer Conversion by Channel")
plt.xlabel("Marketing Channel")
plt.ylabel("Conversion Rate")
plt.grid(axis="y", alpha=0.3)
plt.show()
```



💡 Insight:

Shows lead quality, not just volume

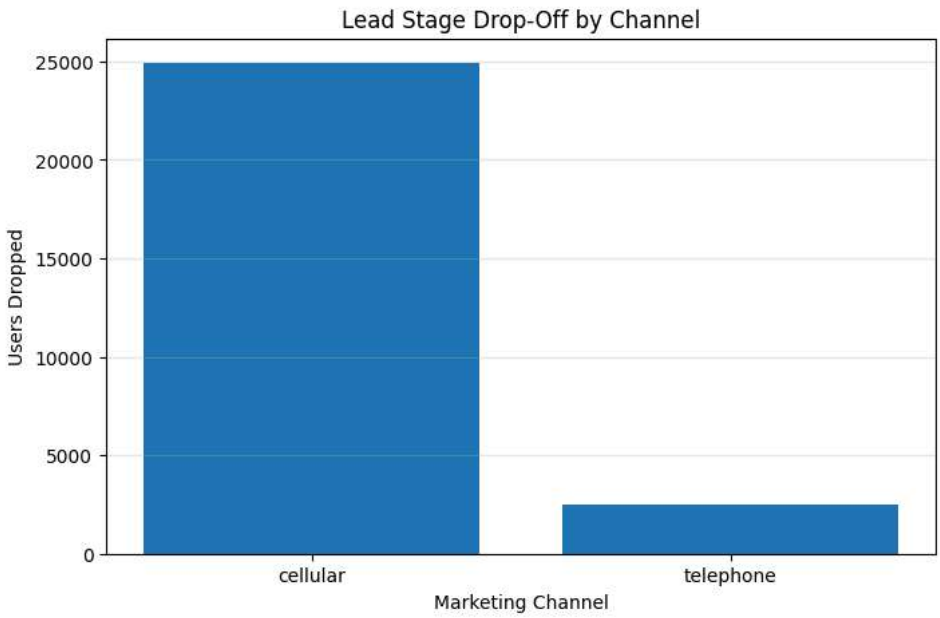
Channels with fewer leads but higher conversion stand out

Used to reallocate marketing budget

Chart: Channel Drop-Off Analysis

```
channel_perf["drop_after_lead"] = channel_perf["leads"] - channel_perf["customers"]

plt.figure(figsize=(8,5))
plt.bar(channel_perf.index, channel_perf["drop_after_lead"])
plt.title("Lead Stage Drop-Off by Channel")
plt.xlabel("Marketing Channel")
plt.ylabel("Users Dropped")
plt.grid(axis="y", alpha=0.3)
plt.show()
```



💡 Insight:

Identifies channels generating low-intent leads

Strong input for campaign refinement

Often used by sales + marketing jointly

Chart: Channel × Funnel Stage Comparison

```
channel_perf[["visitors", "leads", "customers"]].plot(
    kind="bar",
    figsize=(10,6)
)

plt.title("Channel-wise Funnel Stage Comparison")
plt.xlabel("Marketing Channel")
plt.ylabel("Number of Users")
plt.grid(axis="y", alpha=0.3)
plt.show()
```

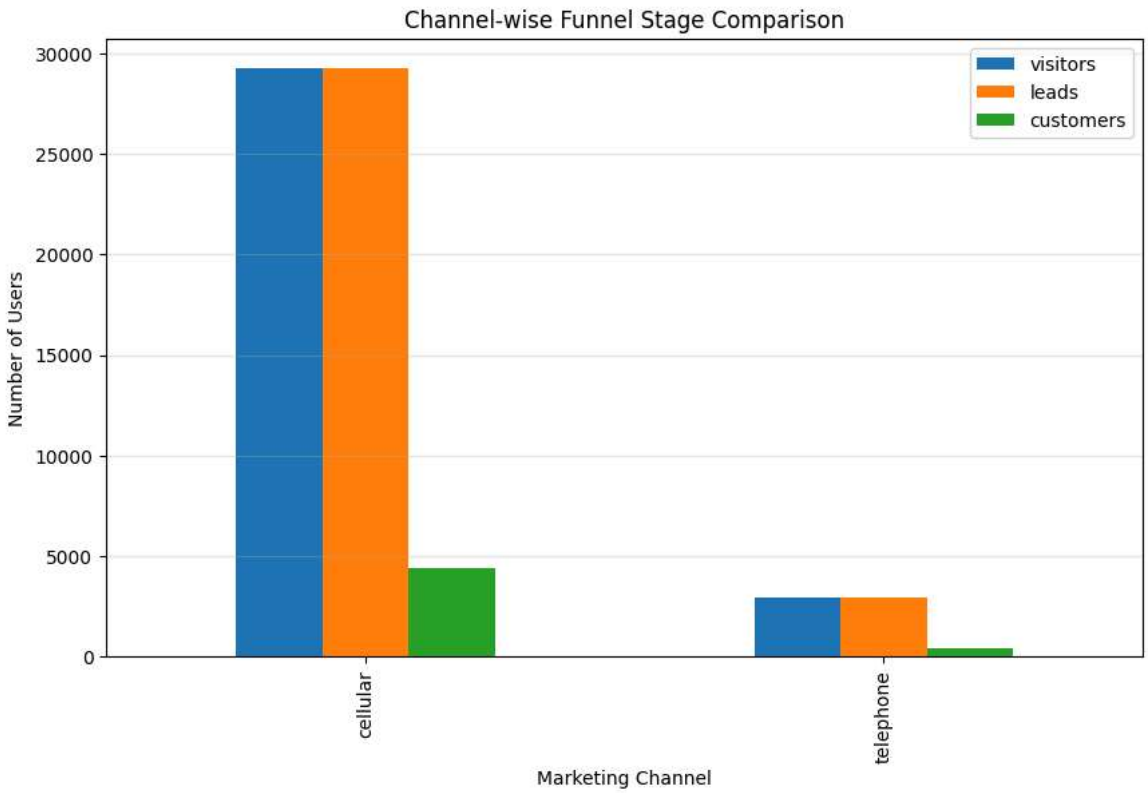


Chart : Monthly Conversion Trend

```
df["date"] = pd.to_datetime(
    df["day"].astype(str) + "-" + df["month"],
    format="%d-%b",
    errors="coerce"
)
```

```
monthly_trend = (
    df.groupby(df["date"].dt.month)
      .agg(customers=("is_customer", "sum"))
)

plt.figure(figsize=(8,5))
plt.plot(monthly_trend.index, monthly_trend["customers"], marker="o")
plt.title("Monthly Customer Conversion Trend")
plt.xlabel("Month")
plt.ylabel("Customers Converted")
plt.grid(alpha=0.3)
plt.show()
```

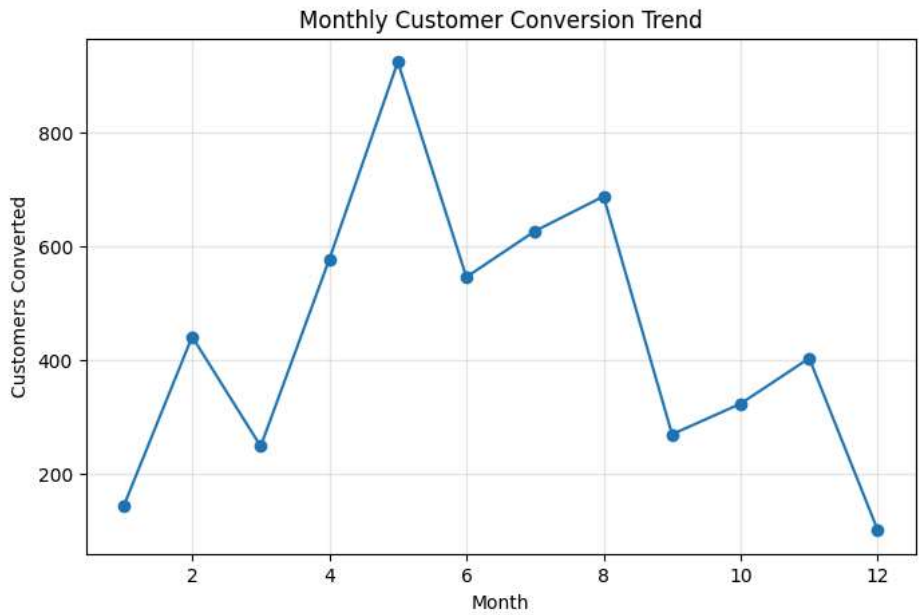


Chart : Channel-wise Conversion Heatmap

```
channel_stage_matrix = channel_perf[[
    "lead_conversion",
    "customer_conversion"
]]

plt.figure(figsize=(8,5))
sns.heatmap(channel_stage_matrix, annot=True, fmt=".2%", cmap="RdYlGn")
plt.title("Channel-wise Funnel Conversion Heatmap")
plt.show()
```

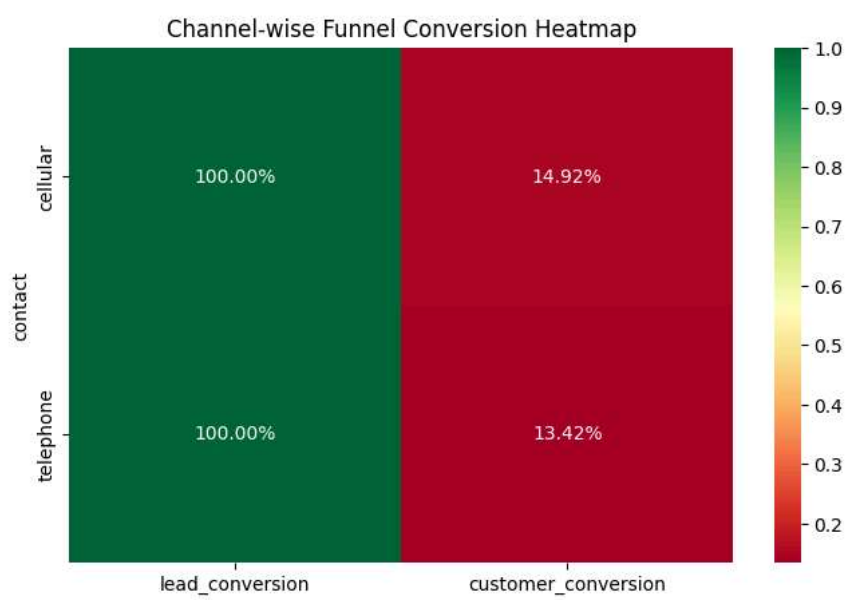


Chart: Pareto Analysis of Lead-to-Customer Drop-offs by Channel

```
dropoffs = channel_perf["leads"] - channel_perf["customers"]
dropoffs = dropoffs.sort_values(ascending=False)

cum_pct = dropoffs.cumsum() / dropoffs.sum()

fig, ax1 = plt.subplots(figsize=(9,5))
```

```
ax1.bar(dropoffs.index, dropoffs, color="steelblue")
ax2 = ax1.twinx()
ax2.plot(dropoffs.index, cum_pct, color="red", marker="o")

ax1.set_ylabel("Users Dropped")
ax2.set_ylabel("Cumulative % Drop-off")
ax2.axhline(0.8, color="gray", linestyle="--")

plt.title("Pareto Analysis of Funnel Drop-offs")
plt.xticks(rotation=45)
plt.show()
```

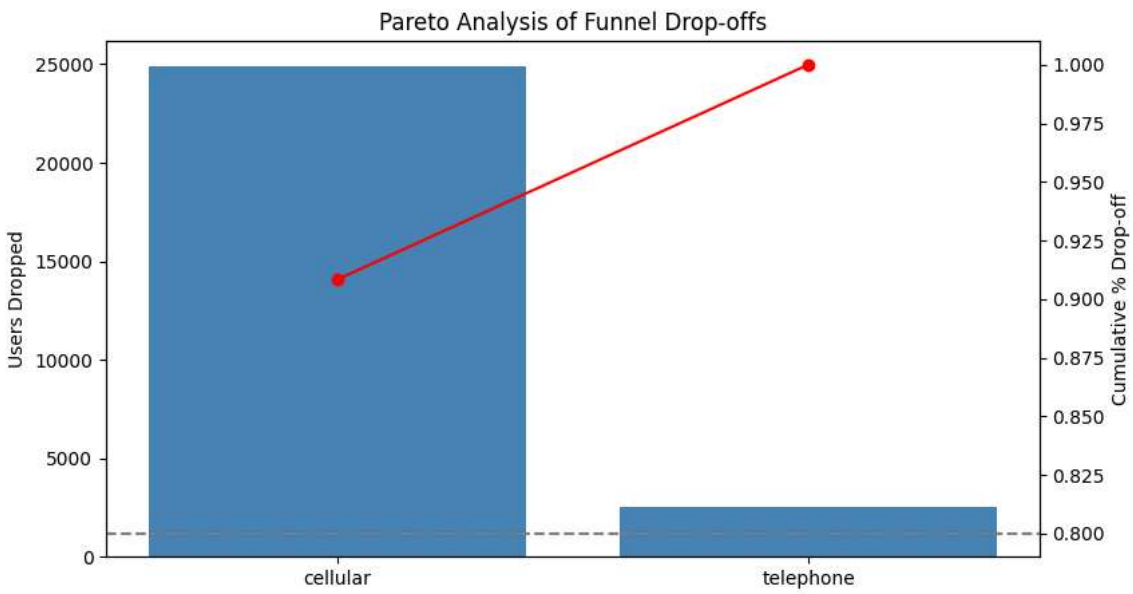


Chart : Segmentation Analysis (Age Group)

```
df["age_group"] = pd.cut(
    df["age"],
    bins=[18,30,45,60,100],
    labels=["18–30","31–45","46–60","60+"]
)

age_perf = df.groupby("age_group")["is_customer"].mean()

plt.figure(figsize=(8,5))
plt.bar(age_perf.index.astype(str), age_perf.values)
plt.title("Customer Conversion by Age Group")
plt.ylabel("Conversion Rate")
```